

PROJECT SYNOPSIS

ON

“WEBSITE HOSTING ON AWS USING APPLICATION LOAD BALANCER AND CLOUDFORMATION”

Project Type: AWS Cloud / Cloud Computing Project

Submitted By

S. No.	Name	Enrollment / Roll Number
1	Mohit Pachauri	12515990022
2	Soujanya Maity	2415001591
3	Ayush Agrawal	2415800024
4	Ishaan Varshney	2415000703

Department of Computer Science and Engineering
Bachelor of Technology (B.Tech)

1. Introduction

Cloud computing has changed the way modern web applications are hosted, making it possible to build infrastructure that is reliable, secure, and available across multiple physical locations without owning any physical hardware. This project, titled “Website Hosting on AWS using Application Load Balancer and CloudFormation,” demonstrates the design and deployment of a highly available website hosting architecture on Amazon Web Services (AWS).

The project uses two EC2 instances running the Apache HTTP Server, placed in two separate Availability Zones, and distributes incoming HTTP traffic between them using an Application Load Balancer (ALB). The entire infrastructure, including networking, security, compute, and load balancing resources, is defined and deployed using an AWS CloudFormation template written in YAML. This approach automates the provisioning process and ensures that the infrastructure can be created consistently and repeatably.

The project focuses on core AWS networking and compute concepts such as Virtual Private Cloud (VPC) design, subnetting across Availability Zones, security groups, load balancing, and infrastructure as code, providing practical, hands-on exposure to how production-style web infrastructure is designed on the cloud.

2. Problem Statement

A website hosted on a single server suffers from two major limitations. First, if that single server fails, becomes overloaded, or is taken down for maintenance, the website becomes completely unavailable, since there is no other server to handle incoming requests. Second, a single server has no way to distribute incoming traffic, so all requests are processed by one machine regardless of its capacity.

In addition, manually creating and configuring cloud resources such as networks, security groups, servers, and load balancers through the AWS Console is time-consuming and prone to human error, and it becomes difficult to reproduce the same environment consistently.

This project addresses these problems by hosting the website on two EC2 instances placed in two different Availability Zones and by using an Application Load Balancer to distribute traffic

between them, so that the website continues to function even if one server or one Availability Zone becomes unavailable. The entire setup is automated using AWS CloudFormation to eliminate manual configuration errors and to make deployment repeatable.

3. Objectives

The objectives of this project are as follows:

- To host a website on AWS EC2 instances running Amazon Linux 2023 and Apache HTTP Server.
- To deploy the web servers across two different Availability Zones for fault tolerance.
- To use an Application Load Balancer to distribute incoming HTTP traffic between the two web servers.
- To configure health checks on the Target Group so that traffic is routed only to healthy instances.
- To implement network security using separate Security Groups for the Application Load Balancer and the EC2 instances.
- To automate the complete deployment of the AWS infrastructure using an AWS CloudFormation template.
- To demonstrate the concepts of high availability, fault tolerance, reliability, and basic scalability in a practical cloud hosting scenario.

4. Technologies / AWS Services Used

The project uses only the AWS services and technologies that are actually implemented in the CloudFormation template and the deployed architecture. These are listed below.

S. No.	Service / Technology	Purpose in the Project
1	Custom VPC	Provides an isolated virtual network for all project resources.
2	Internet Gateway	Allows communication between the VPC and the internet.

S. No.	Service / Technology	Purpose in the Project
3	Public Subnets (2, across 2 AZs)	Host the ALB and EC2 instances across two Availability Zones.
4	Public Route Table	Routes internet-bound traffic from the subnets through the Internet Gateway.
5	ALB Security Group	Controls inbound access to the Application Load Balancer.
6	EC2 Security Group	Controls inbound access to the EC2 web servers.
7	EC2 Instances (2)	Host the Apache web server and serve website content.
8	Application Load Balancer (ALB)	Distributes incoming HTTP traffic to the EC2 instances.
9	ALB Target Group	Groups the EC2 instances and performs health checks on them.
10	ALB Listener	Listens for HTTP traffic on port 80 and forwards it to the Target Group.
11	AWS CloudFormation	Automates provisioning of all the above resources using a YAML template.
12	Amazon Linux 2023	Operating system used on both EC2 instances.
13	Apache HTTP Server (httpd)	Web server software used to serve the website.

5. Proposed System

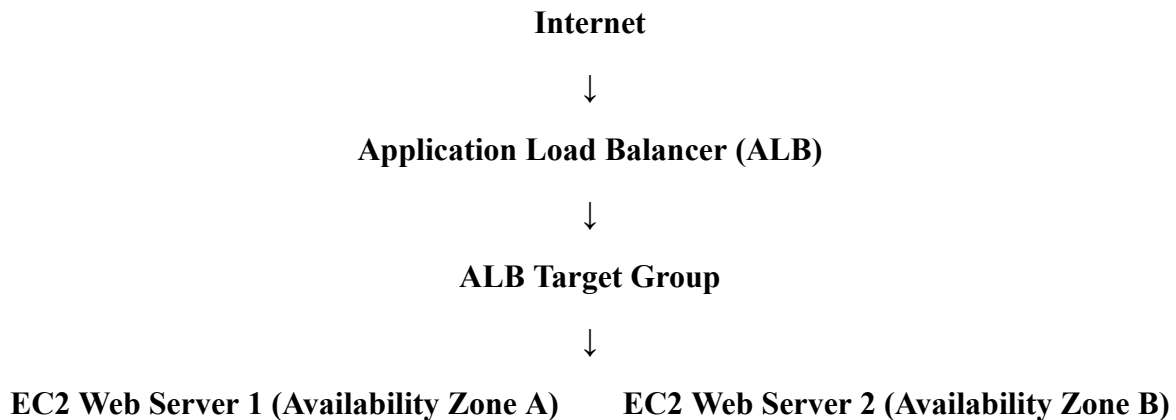
The proposed system hosts a website using two EC2 instances that run identical Apache web servers but are placed in two different Availability Zones within a single AWS Region. Instead of exposing the EC2 instances directly to the internet, all incoming HTTP requests are routed through an internet-facing Application Load Balancer.

The ALB is deployed across both public subnets and forwards traffic to a Target Group that contains both EC2 instances. The Target Group performs health checks on each instance, and the ALB sends traffic only to instances that are reported healthy. Each of the two web servers is configured, through the CloudFormation UserData script, to display a distinct HTML page identifying itself, so that repeated access to the website through the ALB can be used to observe traffic being served by different servers.

The complete set of AWS resources required for this architecture, including the VPC, subnets, route table, Internet Gateway, security groups, EC2 instances, Target Group, ALB, and Listener, is defined in a single AWS CloudFormation YAML template. Deploying this template through the AWS Management Console automatically creates and configures every resource in the correct order, and the stack reaches the `CREATE_COMPLETE` status once all resources are successfully provisioned.

6. System Architecture

The system architecture follows a simple and standard high-availability web hosting pattern. The flow of a client request through the system is illustrated below:



An internet request first reaches the Application Load Balancer, which is deployed as an internet-facing load balancer across two public subnets located in two different Availability Zones. The ALB Listener, configured on port 80, receives the HTTP request and forwards it to the ALB Target Group according to the configured routing rule.

The Target Group contains both EC2 web server instances — Web Server 1 in Availability Zone A and Web Server 2 in Availability Zone B — and continuously performs health checks on them. Based on the health check results, the Target Group forwards the request to one of the two healthy EC2 instances, which then serves the corresponding HTML webpage back to the client through the ALB.

Placing the two web servers in separate Availability Zones ensures that the website remains reachable even if one Availability Zone experiences an outage, since the ALB will continue routing traffic to the healthy instance in the other zone.

7. Methodology / Implementation

The implementation of the project was carried out entirely through a single AWS CloudFormation YAML template, which was deployed using the AWS Management Console. The methodology can be broken down into the following stages.

7.1 Network Configuration

A custom VPC was created to host all the project resources in an isolated network environment. An Internet Gateway was created and attached to this VPC to allow communication with the internet. Two public subnets were created in two different Availability Zones to support high availability. A public Route Table was created and associated with both subnets, with a route directing internet-bound traffic (0.0.0.0/0) to the Internet Gateway, making both subnets publicly reachable.

7.2 Security Configuration

Two separate security groups were defined to control network access. The ALB Security Group allows inbound HTTP traffic on port 80 from the internet (0.0.0.0/0), since the load balancer must be publicly reachable. The EC2 Security Group allows inbound HTTP traffic on port 80 only from the ALB Security Group, which means the EC2 instances do not accept traffic directly from the internet and can only be reached through the Application Load Balancer.

7.3 EC2 Deployment

Two EC2 instances running Amazon Linux 2023 were launched using the CloudFormation template — one in each of the two public subnets, placing them in two different Availability Zones. Each instance was launched with the EC2 Security Group attached, so that it only accepts HTTP traffic originating from the ALB.

7.4 Apache Web Server Configuration

Each EC2 instance was configured using a CloudFormation UserData script that runs automatically at launch. The script installs the Apache HTTP Server (httpd), starts the httpd service, enables it so that it starts automatically on reboot, and creates a simple HTML webpage. The HTML content on each server was made distinct, so that each web server identifies itself, allowing the two servers to be visually distinguished when accessed through the load balancer.

7.5 Target Group Configuration

An ALB Target Group was created and configured to use the HTTP protocol on port 80. Both EC2 instances were registered as targets within this Target Group. Health checks were configured on the Target Group so that the ALB can determine whether each EC2 instance is capable of serving the website correctly.

7.6 Application Load Balancer Configuration

An internet-facing Application Load Balancer was created and deployed across the two public subnets so that it can operate across both Availability Zones. The ALB Security Group was attached to the load balancer to control the traffic permitted to reach it.

7.7 Listener Configuration

An ALB Listener was configured on port 80 using the HTTP protocol. The listener's default action forwards all incoming requests to the ALB Target Group, which then routes each request to one of the healthy EC2 instances.

7.8 Health Checks

Health checks were configured on the Target Group to periodically verify that each EC2 instance is responding correctly to HTTP requests. Only instances that pass the health check are considered by the ALB when routing incoming traffic, which prevents requests from being sent to a server that is not functioning correctly.

7.9 CloudFormation Deployment

The complete infrastructure — the VPC, Internet Gateway, subnets, route table, security groups, EC2 instances, Target Group, ALB, and Listener — was defined in a single YAML CloudFormation template. This template was uploaded and deployed as a stack through the AWS

Management Console. CloudFormation created all the defined resources automatically and in the correct dependency order, and the stack reached the `CREATE_COMPLETE` status upon successful deployment. The ALB's DNS name was configured as a stack output, providing a direct URL through which the hosted website can be accessed.

8. Security Design

The security design of this project is based on the principle of restricting direct access to the EC2 web servers and only exposing the Application Load Balancer to the internet. This is achieved using two layered security groups.

Security Group	Inbound Rule	Purpose
ALB Security Group	Allow HTTP (port 80) from 0.0.0.0/0	Allows the internet-facing load balancer to receive public HTTP traffic.
EC2 Security Group	Allow HTTP (port 80) only from the ALB Security Group	Ensures EC2 instances can be reached only through the ALB, not directly from the internet.

Because the EC2 Security Group only permits traffic originating from the ALB Security Group, the two web servers are not directly accessible from the public internet. All website traffic must pass through the Application Load Balancer, which acts as the single controlled entry point into the system. This design reduces the exposure of the EC2 instances and reflects a standard security practice for load-balanced web architectures on AWS.

9. Working of the System

When a user requests the website using the ALB's DNS name, the request first reaches the Application Load Balancer through the ALB Listener on port 80. The ALB checks the current health status of the registered targets in the Target Group and forwards the request to one of the healthy EC2 instances — either Web Server 1 in Availability Zone A or Web Server 2 in Availability Zone B.

The selected EC2 instance, running Apache HTTP Server, responds with the HTML webpage that was created during instance launch through the CloudFormation UserData script. Since each server hosts a distinctly identifiable webpage, refreshing the website multiple times demonstrates

that different requests are being served by different EC2 instances, confirming that the load balancer is distributing traffic across both servers.

If one EC2 instance becomes unhealthy, the Target Group's health check marks it as unhealthy, and the ALB stops routing traffic to it, sending all subsequent requests to the remaining healthy instance instead. This demonstrates the fault-tolerant behaviour of the architecture.

10. Deployment and Verification

The CloudFormation template was deployed by uploading it as a new stack in the AWS Management Console. During deployment, CloudFormation provisioned the VPC, Internet Gateway, subnets, route table, security groups, EC2 instances, Target Group, ALB, and Listener in the required dependency order.

Deployment was verified by observing the stack status in the CloudFormation console until it reached `CREATE_COMPLETE`, confirming that every resource in the template was created successfully without errors. The health status of both EC2 instances was verified in the Target Group's target health view, confirming that both instances were registered and marked healthy.

The website was then accessed using the ALB's DNS name, which was made available as a CloudFormation stack output. Accessing this DNS name in a browser and refreshing the page multiple times confirmed that requests were being served by both EC2 instances in turn, verifying that the load balancer was correctly distributing traffic.

11. Results

The CloudFormation stack was deployed successfully and reached the `CREATE_COMPLETE` status, confirming that all defined AWS resources — the VPC, subnets, route table, Internet Gateway, security groups, EC2 instances, Target Group, ALB, and Listener — were created correctly.

Both EC2 instances were registered in the Target Group and reported as healthy by the configured health checks. Accessing the website through the ALB's DNS name successfully returned the hosted webpage, and repeated access confirmed that the Application Load Balancer

distributed requests between Web Server 1 and Web Server 2, each identifying itself with a distinct webpage.

These results confirm that the objectives of the project — automated infrastructure deployment, distributed traffic handling across two Availability Zones, and controlled access to the EC2 instances through security groups — were successfully achieved.

12. Advantages

- The website remains available even if one EC2 instance or one Availability Zone becomes unhealthy, since the ALB routes traffic to the remaining healthy instance.
- The Application Load Balancer distributes incoming traffic across two web servers, avoiding dependence on a single server.
- The layered security group design prevents direct public access to the EC2 instances, restricting inbound traffic to only the ALB.
- Health checks ensure that traffic is not routed to an unhealthy or unresponsive server.
- The use of AWS CloudFormation automates the entire deployment process, removing the need for manual, repetitive, and error-prone resource creation.
- The infrastructure defined in the CloudFormation template can be redeployed consistently, making the setup reproducible.

13. Limitations / Scope for Future Enhancement

The current implementation is limited to the specific AWS services and configurations described in this synopsis. The following points describe its limitations and realistic areas for future enhancement, without assuming the addition of features that have not been implemented in this project.

- The number of EC2 instances is fixed at two and is not automatically adjusted based on traffic load, since Auto Scaling has not been implemented in this project.
- The website is served only over HTTP on port 80; HTTPS/SSL encryption has not been configured.

- The website content is a static HTML page created through the UserData script; no database or dynamic backend has been implemented.
- Access to the website currently relies on the ALB's default DNS name, as a custom domain name service has not been configured.
- As a future enhancement, Auto Scaling could be introduced to automatically adjust the number of EC2 instances based on demand.
- As a future enhancement, HTTPS could be configured on the ALB Listener to secure traffic between clients and the load balancer.
- As a future enhancement, a custom domain name could be mapped to the ALB's DNS name to provide a more user-friendly website address.

14. Conclusion

This project successfully demonstrates the design and deployment of a highly available website hosting architecture on AWS using an Application Load Balancer and AWS CloudFormation. By placing two Apache-based EC2 web servers across two Availability Zones and routing all traffic through an Application Load Balancer with configured health checks, the project achieves fault tolerance, distributed traffic handling, and controlled security access through layered security groups.

The use of AWS CloudFormation to automate the entire deployment process highlights the benefits of infrastructure as code, enabling consistent, repeatable, and error-free provisioning of cloud resources. Overall, the project provides practical, hands-on experience with core AWS networking, compute, security, load balancing, and automation concepts that form the foundation of production-grade cloud architectures.

15. References

- AWS Documentation – Amazon Virtual Private Cloud (VPC) User Guide, Amazon Web Services.
- AWS Documentation – Amazon EC2 User Guide for Linux Instances, Amazon Web Services.

- AWS Documentation – Application Load Balancers, Elastic Load Balancing User Guide, Amazon Web Services.
- AWS Documentation – AWS CloudFormation User Guide, Amazon Web Services.
- Apache HTTP Server Documentation, The Apache Software Foundation.